

Project Summary: Disaster Tweet Classifier

1. Core Architecture & Rationale

I built a dual-interface system (Streamlit Web UI + CLI batch predictor) that identifies whether a tweet describes a real-world emergency.

The engine uses a **TF-IDF Vectorizer** coupled with a **Logistic Regression** model. While deep-learning Transformers (like BERT) or LLM APIs are common choices, this statistical approach was chosen for its practical advantages:

- **Zero Infrastructure Friction:** It requires no GPU, incurs zero API token costs, and trains locally in under two seconds.
- **Minimal Footprint:** The entire compiled model asset is tiny, ensuring the application can be spun up instantly on any laptop.
- **Strong Baseline:** It achieves a solid ~0.78–0.80 validation F1-score, providing a highly efficient performance-to-overhead ratio.

2. Validation & Iterative Modifications

Using an AI collaborator for initial scaffolding, the implementation was refined through several manual validation steps to ensure production readiness:

- **Dynamic File Paths:** Changed the original path structure to dynamic relative paths (`os.path.abspath`). This ensures that whether you run the scripts from your home directory or a reviewer clones it fresh on their machine, file loading and saving never crash.
- **Strict Parameter Alignment:** Corrected scikit-learn splitting arguments to fix a parameter typo (`test_size`).
- **CLI Conformance:** Rewrote the batch script to explicitly support `--input` and `--output` flags using `argparse`, allowing seamless evaluation of any held-out dataset.
- **Clean-Room Testing:** Successfully validated the repository inside an isolated, clean virtual environment (`venv`) to ensure pinned dependencies in `requirements.txt` deploy without conflict.

3. Limitations & Next Steps

- **Contextual Weakness:** TF-IDF relies on explicit word counts. It easily generates false positives on figurative language or internet sarcasm (e.g., *"My grading queue is a complete trainwreck, I'm dying"*).
- **Metadata Ignored:** The model skips valuable tweet features like geographical location data, hashtags, and mentions.

Future Enhancements: Given more time, the system would be upgraded to **DistilBERT** to inject deeper semantic and contextual understanding while keeping local CPU execution fast and practical.

Annex:

AI tool (Gemini Flash 3.5) was used to help trouble shoot errors I was having in the code and the set up the environment. As I was not using my personal computer, I had problems with multiple python versions in vscode and in the terminal. AI was also used in generating the readme file and the project report.